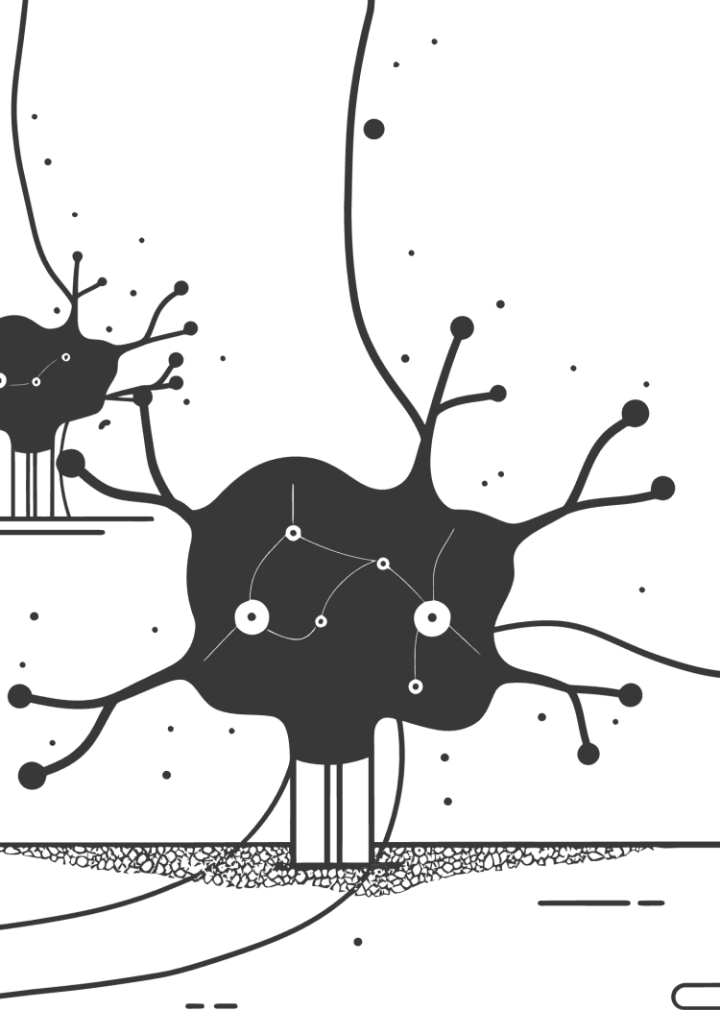




Advanced Interactions & Configuration

Session 4 · Playwright Automation Bootcamp

CONFIG

SETUP

INTERACTIONS

HOOKS

TEARDOWN

Dialogs



Alert
OK



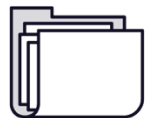
Confirm
OK-Cancel



Prompt
Input + OK/Cancel

Describe & Lifecycle Hooks

Named container for a group of related tests



describe()



Test

Test

Test

```
test('Check thing 1', async ({ page }) => {  
  // Set up  
  // ...  
  // Clean up  
});
```

```
test('Check thing 2', async ({ page }) => {  
  // Set up  
  // ...  
  // Clean up  
});
```

```
test.beforeEach('Inject state', ...);

test('Check thing 1', async ({ page }) => {
  // ...
});

test('Check thing 2', async ({ page }) => {
  // ...
});

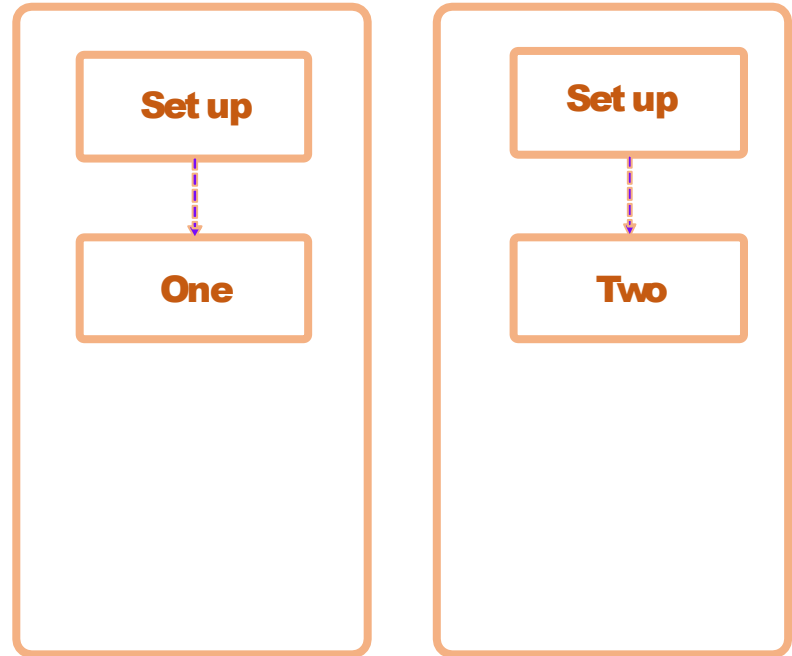
test.afterEach('Shut down DB', ...);
```

“A ‘beforeAll’ hook is executed once per worker process before all tests.”

```
test.beforeAll('Set up', ...)
```

```
test('One', ...);
```

```
test('Two', ...);
```



Harnessing the power of Playwright's Test Runner

Workers

Workers



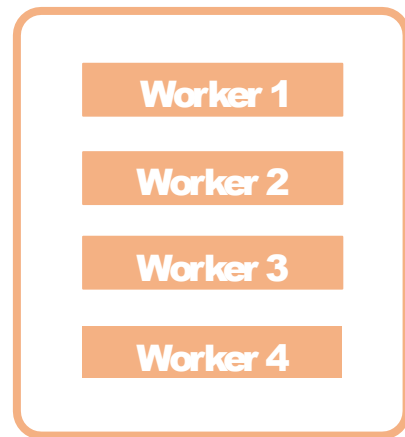
With `"fullyParallel: true"`

1 test - 1 worker

2 tests - 2 workers

6 or 20 tests - 4 workers

Defaults to half of the number of
logical CPU cores



Worker 1

Worker 2

Worker 3

Worker 4

Base speed: 3.00 GHz

Sockets: 1

Cores: 8

Logical processors: 8

```
npx playwright test --workers 8
```

```
export default defineConfig({  
  
  workers: 8  
  
});
```

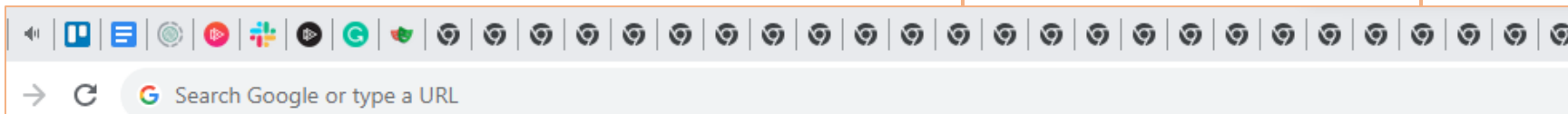
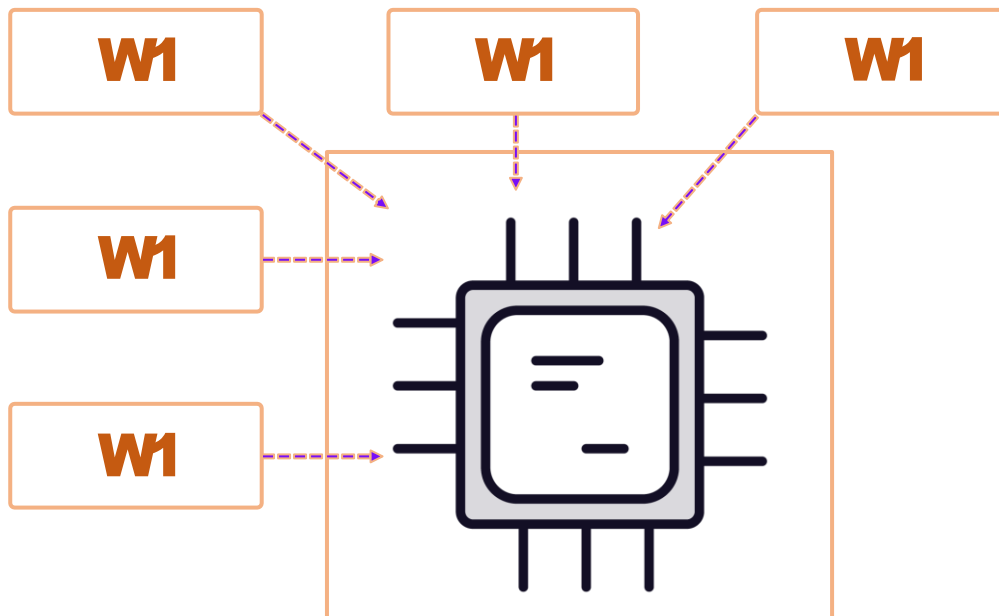
```
npx playwright test --workers 8
```

```
export default defineConfig({  
  workers: 50  
});
```

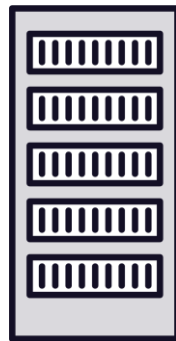


Resource Contention

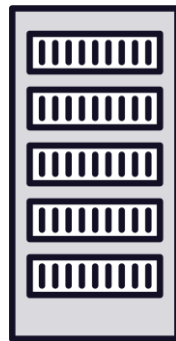
workers: 5



```
export default defineConfig({  
  workers: 4  
});
```




```
export default defineConfig({  
  workers: 50%  
});
```



```
test.use()
```

```
test.use({  
  viewport: { width: 1200, height: 400 },  
  baseURL: 'your/url',  
  locale: 'en-US',  
  headless: false,  
  javascriptEnabled: false,  
  acceptDownloads: true,  
  colorScheme: 'light',  
  geolocation: ...  
});
```

Configuring Playwright Tests

Order of Configuration. Global, Group and test configuration

```
await btn.click({ clickCount: 5 });
```

```
test.use({  
  locale: 'en-US',  
  headless: false,  
  // ...  
});
```

```
export default defineConfig({

  testDir: './tests',
  fullyParallel: true,
  // ...

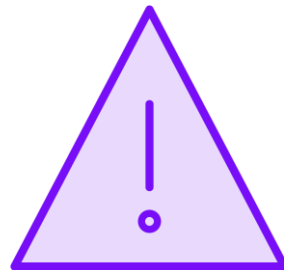
  use: {
    baseURL: 'http://localhost:3000',
    screenshot: 'only-on-failure',
    // ...
  },

  // ...
})
```

Global Configuration

Group Configuration

Test Configuration



More specific config
wins in case of conflict

Global Configuration

locale: 'en-US'

Group Configuration

Test Configuration

locale: 'es-ES'

Playwright CodeGen

Let Playwright write the first draft — record interactions in a live browser and get real test code instantly.

What Is CodeGen?



How It Works

CodeGen launches a real browser with an inspector panel. Every click, fill, and navigation is translated into Playwright code in real time.



Use it to **bootstrap** new tests
— not as a finished product.

Start Recording Against TestMart

Launch CodeGen pointed at your local app:

```
npx playwright codegen http://localhost:3000
```

A browser window opens alongside the inspector — interact with the page and watch code appear live.

Generated Code Example — Login Flow

Recording a login produces locators and interactions automatically:

```
await page.goto('http://localhost:3000/login');
await page.getByTestId('login-email').fill('standard_user@example.com');
await page.getByTestId('login-password').fill('Password123!');
await page.getByTestId('login-submit').click();
await expect(page.getByTestId('nav-username')).toBeVisible();
```

Using CodeGen Correctly

1

Record the Happy Path

Get locators and flow right from real interactions.

2

Copy Into a Test File

Move generated code into a proper `.spec.ts` file.

3

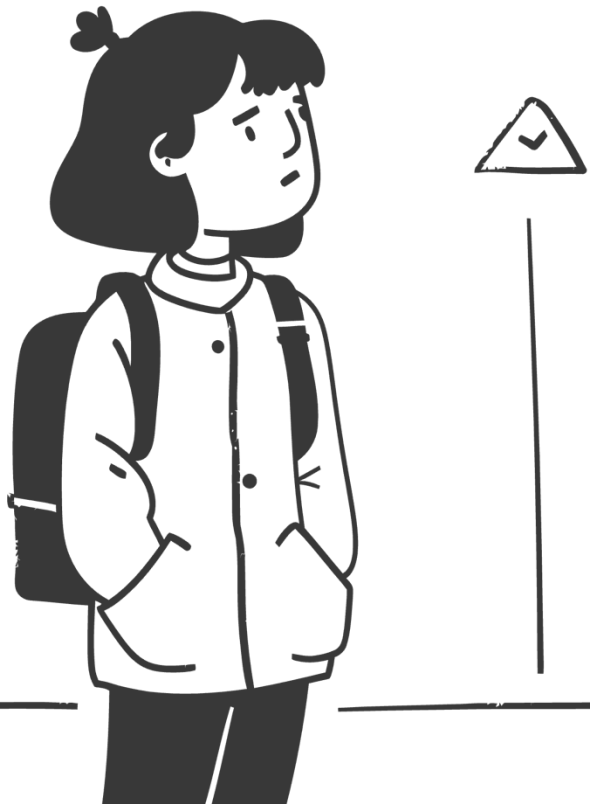
Refactor Into POM

Move repeated actions into Page Object Model classes.

4

Add Assertions

CodeGen only records clicks and fills — add what the page *should* show.



The Key Limitation

Generated code is a starting point — it produces raw interactions, not maintainable tests. Always refactor into POM classes before committing.

Skip the Login Screen When Recording

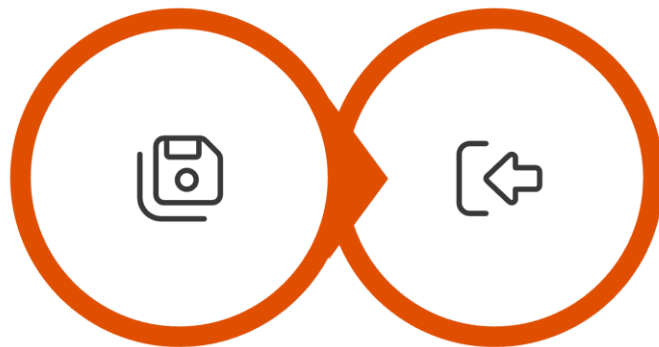
The Problem

Most TestMart pages require authentication. Without help, CodeGen forces a manual login at the start of every session — wasted steps that pollute generated code.

The Solution

A two-step workflow using `--save-storage` and `--load-storage` lets you record login once and reuse the session for every subsequent recording.

Two-Step Auth Workflow



Save Session

Load Session

Record login as `standard_user@example.com` / `Password123!` once, then reuse `auth.json` for every feature you record that day.

The Commands

Step 1 — Save the session:

```
npx playwright codegen \  
  --save-storage=auth.json \  
  http://localhost:3000/login
```

Step 2 — Load the session:

```
npx playwright codegen \  
  --load-storage=auth.json \  
  http://localhost:3000/products
```

Protect Your Session Tokens

⊗ **Add `auth.json` to `.gitignore`** — it contains live session tokens that must never be committed.

```
# .gitignore
auth.json
playwright/.auth/
```

When It Matters Most

Any flow starting on a protected page — cart, checkout, order history, account settings.

Record login once per session; reuse saved state for every feature you record that day.